

MODUL 210 · PUBLIC CLOUD FÜR ANWENDUNGEN NUTZEN

CI/CD mit GitHub Actions

Branches, Pull-Requests & eine automatisierte Build-/Test-Pipeline

Autor: Philipp Seewer **Schule:** WISS **Projekt:** Steam Library (M223) **Datum:** 13.06.2026

1 Worum geht es?

Im Modul 210 lernen wir, **wie professionelle Software-Teams arbeiten**. Statt Code direkt in den Hauptstand zu schreiben und zu hoffen, dass alles funktioniert, nutzt man zwei Werkzeuge, die zusammenspielen:

- **Branches & Pull-Requests** – damit der Hauptstand (`main`) immer stabil bleibt.
- **GitHub Actions** – eine Pipeline, die bei jedem Push automatisch testet und baut.

Ziel dieser Zusammenfassung: erklären, **was** die Aufgaben sind, **warum** man das so macht und **was wir konkret umgesetzt haben**.

2 Aufgabe 1 – Branches & Pull-Requests

Ein **Branch** ist ein „Seitenast“ des Projekts. Man arbeitet darin an einer Änderung, ohne den Hauptstand `main` anzufassen. Ist die Änderung fertig, stellt man einen **Pull-Request** (PR): eine Anfrage, die Änderungen aus dem Branch in `main` zu übernehmen. Der PR kann vorher geprüft, getestet und besprochen werden – erst dann wird **gemergt** (zusammengeführt).

Warum so? Direkt in `main` zu pushen ist nur für sehr kleine Projekte okay. Mit Branches & Pull-Requests bleibt `main` **immer einsatzfähig (deploybar)** – nichts Ungetestetes landet im Hauptstand.

Der typische Ablauf (genau so haben wir es gemacht):

```
# 1) Eigenen Branch erstellen und hineinwechseln
git checkout -b ci/github-actions

# 2) Änderungen committen
git commit -m "ci: GitHub Actions Workflow ..."

# 3) Branch online stellen (push)
git push -u origin ci/github-actions

# 4) Pull-Request auf GitHub öffnen und nach Prüfung mergen
```

3 Aufgabe 2 – GitHub Actions (CI/CD)

GitHub Actions ist eine automatische Pipeline. Sie wird durch eine `.yaml`-Datei im Ordner `.github/workflows/` gesteuert und läuft **bei jedem Push und Pull-Request** auf einem neutralen Linux-Server (Ubuntu) bei GitHub. Sie übernimmt drei Dinge:

- **Testen** – alle automatischen Tests laufen automatisch.
- **Bauen** – das Projekt wird kompiliert/gebaut.
- **Artefakt speichern** – das fertige Programm (die `.jar`-Datei) wird abgelegt.

Warum so? Fehler werden sofort sichtbar – **bevor** gemergt wird. Und weil immer dieselbe Server-Umgebung verwendet wird, gibt es kein „Auf meinem Rechner lief es aber!“ mehr.

4 Unsere Umsetzung

4.1 Die Pipeline – `.github/workflows/ci.yaml`

Eine Workflow-Datei mit zwei unabhängigen Jobs, die parallel laufen:

Job	Was er tut
backend (Maven)	JDK 21 (Temurin) einrichten → <code>mvn clean test</code> (JUnit + MockMvc) → <code>mvn package -DskipTests</code> → die <code>.jar</code> als Artefakt hochladen.
frontend (Vitest)	Node.js 22 einrichten → <code>npm install</code> → <code>npm run test</code> (Vitest) → <code>npm run build</code> .

Zwei Besonderheiten unseres Projekts: (1) Die Integrationstests laufen hermetisch gegen eine In-Memory-Datenbank (**H2**) – darum braucht die CI **kein MySQL**. (2) Wir verwenden `npm install` statt `npm ci`, weil das `package-lock.json` in `.gitignore` steht (und `npm ci` ein Lockfile bräuchte).

4.2 Anpassung am Dockerfile

Vorher hat der Docker-Image-Build die Tests selbst ausgeführt (`mvn clean test package`). Das haben wir geändert auf:

```
RUN mvn -B clean package -DskipTests
```

Warum? Die Tests laufen jetzt in der CI-Pipeline. Sie im Image-Build zu wiederholen wäre doppelte Arbeit und würde den Build unnötig verlangsamen.

4.3 Branch, Commit & Push

Beide Dateien (`ci.yaml` neu, `Dockerfile` geändert) wurden auf dem Branch `ci/github-actions` committet und zu GitHub gepusht – also **nicht** direkt in `main`, genau wie es die Methode vorsieht.

4.4 Lokale Verifikation

Bereich	Ergebnis
Backend	✅ 16 Tests grün (8 GameService · 3 GameLibraryService/Mockito · 5 GameController/MockMvc).
Frontend	⚠️ Lokal nicht ausführbar (<code>esbuild</code> -Problem unter OneDrive/Windows). Kein CI-Problem – der Linux-Server der Pipeline lädt die passende Binärdatei frisch.

5 Zuordnung zu den Lehr-Folien

Folie	Lehr-Inhalt	In unserer <code>ci.yml</code>
15	Java CI mit Maven (setup-java, JDK 21, Cache, <code>mvn test</code>)	Job backend
16	Build-Artefakt (<code>package</code> + <code>upload-artifact</code>)	Schritte „JAR bauen“ + „Artefakt speichern“
17	Frontend Node.js CI (setup-node, build, test)	Job frontend
9–13	Branch → Push → Pull-Request → Merge	Branch + Push erledigt; PR folgt

6 Der letzte Schritt: Pull-Request mergen

Damit der professionelle Ablauf vollständig ist, fehlt nur noch das Mergen im Browser:

- 1 Pull-Request öffnen** – auf GitHub Base `main` ← Compare `ci/github-actions` .
- 2 CI abwarten** – im PR erscheinen die Checks **backend** und **frontend**; warten bis beide grün sind.
- 3 Mergen** – „Merge pull request“ → „Confirm“. Die Änderungen sind nun in `main` .

Ergebnis: Mit dem Merge sind **beide Modul-Themen in einem Workflow demonstriert** – Branch/Pull-Request (Block 3) und die automatische GitHub-Actions-Pipeline (Block 4). Der `main` -Branch bleibt dabei jederzeit einsatzfähig.

7 Begriffe kurz erklärt

Begriff	Bedeutung
CI/CD	Continuous Integration / Continuous Delivery – automatisches Testen & Bauen bei jeder Änderung.
Branch	Ein paralleler Entwicklungsstand neben <code>main</code> .
Pull-Request	Anfrage, Änderungen eines Branches in einen anderen (meist <code>main</code>) zu übernehmen.
Merge	Das Zusammenführen der Änderungen in den Zielbranch.
Workflow / Action	Die <code>.yaml</code> -Datei bzw. die automatischen Schritte der Pipeline.
Job	Ein in sich abgeschlossener Teil der Pipeline (z. B. „backend“).
Artefakt	Das gebaute Ergebnis (z. B. die <code>.jar</code> -Datei), das gespeichert wird.
Runner	Der Server, auf dem die Pipeline läuft (hier: Ubuntu bei GitHub).